What is APACHE KAFKA? With Example Why KAFKA ,what is its need, why not database only can handle...| Understand with example(Zomato Live Location Tracking) Important Features Architecture of KAFKA Starting servers (KAFKA and Zookeeper) Play with commands(Creating Topic, Producing Message and Consuming Mesage using Console) Understand Project UseCase (Zomato Live Location Tracking) explain theoritically conceptually how it works and kafka used Creating Two Spring Boot Project(DeliveryBoy and EndUser app) Coding for Producing message in DeliveryBoy Service Coding for Consuming Message in EndUser services Testing App with 200K messages live. for absolute beginner

What is Apache Kafka? (With Example)

Apache Kafka is an **open-source distributed streaming platform** designed to handle large-scale, real-time data feeds. Think of it as a super-efficient messaging system that can process and move massive amounts of data between systems or applications in real time.

- **Simple Analogy**: Imagine Kafka as a "post office" for data. Producers (senders) drop off messages (data), and consumers (receivers) pick them up whenever they're ready. The post office (Kafka) ensures messages are stored, organized, and delivered reliably, even if millions of letters are sent daily.
- **Example**: In a Zomato-like app, when a delivery boy moves, their location updates (e.g., latitude/longitude) are sent to Kafka. The app's backend and the end user's phone can then fetch these updates in real time to show the delivery boy's live location on a map.

Why Kafka? What's Its Need? Why Not Just a Database?

Databases (like MySQL or PostgreSQL) are great for storing and querying data, but they struggle with **real-time, high-throughput data streams**. Here's why Kafka shines:

1. **Scalability**: Kafka can handle millions of messages per second across distributed systems, while databases slow down under such load.
2. **Real-Time Processing**: Kafka processes data as it arrives (streaming), whereas databases are better for static, stored data.
3. **Decoupling**: Producers and consumers don't need to know each other—Kafka acts as a middleman. In a database, apps must directly connect and query it.
4. **Durability**: Kafka stores messages on disk for a configurable time (e.g., 7 days), so consumers can replay or catch up if needed. Databases overwrite or delete data unless explicitly designed otherwise.

Zomato Live Location Example:

- **Problem**: A delivery boy's location updates every 5 seconds. Zomato has 10,000 active deliveries, so that's 2,000 updates/second. A database would choke under this load with constant writes and reads.
- **Kafka Solution**: The delivery boy's app (producer) sends location updates to Kafka. The end user's app (consumer) subscribes to these updates in real time. Kafka handles the load effortlessly, storing updates temporarily so no data is lost even if the consumer lags.

Important Features of Kafka

1. **Publish-Subscribe Model**: Producers publish messages to "topics," and consumers subscribe to those topics.
2. **High Throughput**: Handles millions of messages per second.
3. **Fault Tolerance**: Data is replicated across multiple servers (brokers) so it's not lost if one fails.
4. **Durability**: Messages are stored on disk, not just in memory, for a set retention period.
5. **Scalability**: Add more brokers to handle more data seamlessly.
6. **Distributed System**: Runs across multiple machines for reliability and performance.

Architecture of Kafka

Kafka's architecture consists of several components:

1. **Broker**: A Kafka server that stores and manages data. You can have multiple brokers for scalability and fault tolerance.
2. **Topic**: A category or feed name where messages are published (e.g., "delivery-boy-locations"). Topics are split into **partitions** for parallel processing.
3. **Partition**: A subset of a topic's data. Each partition is stored on a broker and can be replicated.
4. **Producer**: An application that sends messages to a topic.
5. **Consumer**: An application that reads messages from a topic.
6. **Zookeeper**: A separate service Kafka uses to manage its cluster (e.g., track which broker is alive).

Flow in Zomato Example:

- Delivery boy's app (producer) sends location updates to the "delivery-boy-locations" topic.
- The topic has partitions (e.g., 10) spread across brokers.
- The end user's app (consumer) reads from the relevant partition to get live updates.

Starting Kafka and Zookeeper

Kafka relies on Zookeeper for coordination. Here's how to set it up on your local machine (Windows/Mac/Linux):

Prerequisites:

- Java (JDK 8 or higher) installed.
- Download Kafka from apache.org (e.g., kafka_2.13-3.6.0.tgz).

Steps:

1. **Extract Kafka**:
   - Unzip the downloaded file to a folder (e.g., C:\kafka or /home/user/kafka).
2. **Start Zookeeper**:
   - Open a terminal in the Kafka folder.
   - Run:
     text
     CollapseWrapCopy
     ```
     bin/zookeeper-server-start.sh config/zookeeper.properties
     ```
     (Windows: bin\windows\zookeeper-server-start.bat config\zookeeper.properties)
   - Zookeeper runs on port 2181 by default.
3. **Start Kafka Server**:
   - Open a new terminal in the Kafka folder.
   - Run:
     text
     CollapseWrapCopy
     ```
     bin/kafka-server-start.sh config/server.properties
     ```
     (Windows: bin\windows\kafka-server-start.bat config\server.properties)
   - Kafka runs on port 9092 by default.

Play with Commands

Let's create a topic and send/receive messages using Kafka's console tools.

1. **Create a Topic**:
   text
   CollapseWrapCopy
   ```
   bin/kafka-topics.sh --create --topic delivery-boy-locations
   --bootstrap-server localhost:9092 --partitions 3 --replication-factor 1
   ```
   (Windows: bin\windows\kafka-topics.bat ...)
2. **Produce Messages**:

- Open a terminal and start a producer:
  text
  CollapseWrapCopy
  ```
  bin/kafka-console-producer.sh --topic delivery-boy-locations --bootstrap-server localhost:9092
  ```
  (Windows: bin\windows\kafka-console-producer.bat ...)

Type messages like:
text
CollapseWrapCopy
```
{"deliveryId": "D1", "lat": 12.9716, "lon": 77.5946}
```

- `{"deliveryId": "D1", "lat": 12.9720, "lon": 77.5950}`

3. **Consume Messages**:
   - Open another terminal and start a consumer:
     text
     CollapseWrapCopy
     ```
     bin/kafka-console-consumer.sh --topic delivery-boy-locations --bootstrap-server localhost:9092 --from-beginning
     ```
     (Windows: bin\windows\kafka-console-consumer.bat ...)
   - You'll see the messages you typed in the producer.

Zomato Live Location Tracking Use Case (Theoretical)

Concept:

- **Goal**: Track a delivery boy's location in real time and show it to the end user.
- **How Kafka Helps**:
  - Delivery boy's app sends location updates to a Kafka topic (delivery-boy-locations).
  - Each delivery has a unique ID (e.g., "D1"), and updates are partitioned by this ID.
  - The end user's app subscribes to updates for "D1" and shows the live location on a map.

Flow:

1. Delivery boy moves → App sends {deliveryId: "D1", lat: 12.9716, lon: 77.5946} to Kafka.
2. Kafka stores this in the delivery-boy-locations topic.
3. End user's app reads the message and updates the map.

Creating Two Spring Boot Projects

Let's build:

1. **DeliveryBoy App**: Produces location messages.
2. **EndUser App**: Consumes location messages.

Prerequisites:

- Java, Maven, and an IDE (e.g., IntelliJ or Eclipse).
- Kafka running locally.

Step 1: Setup Projects

- Create two Spring Boot projects via start.spring.io:
  - Dependencies: Spring Web, Spring for Apache Kafka.
  - Names: DeliveryBoyApp, EndUserApp.

Step 2: Configure application.properties

For both projects, add to src/main/resources/application.properties:

text
CollapseWrapCopy
```
spring.kafka.bootstrap-servers=localhost:9092
```

Coding for Producing Messages (DeliveryBoyApp)

Add a Producer Class:
java
CollapseWrapCopy
```java
package com.example.deliveryboyapp;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;

@Service
public class LocationProducer {
    @Autowired
    private KafkaTemplate<String, String> kafkaTemplate;

    public void sendLocation(String deliveryId, double lat, double lon) {
```

```
        String message = String.format("{\"deliveryId\": \"%s\", \"lat\":
%.4f, \"lon\": %.4f}", deliveryId, lat, lon);
        kafkaTemplate.send("delivery-boy-locations", deliveryId, message);
        System.out.println("Sent: " + message);
    }
```

1. }

Controller to Trigger Messages:
java
CollapseWrapCopy

```
package com.example.deliveryboyapp;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class LocationController {
    @Autowired
    private LocationProducer producer;

    @GetMapping("/send-location")
    public String sendLocation(
            @RequestParam String deliveryId,
            @RequestParam double lat,
            @RequestParam double lon) {
        producer.sendLocation(deliveryId, lat, lon);
        return "Location sent!";
    }
```

2. }

Coding for Consuming Messages (EndUserApp)

Add a Consumer Class:
java
CollapseWrapCopy

```
package com.example.enduserapp;
```

```java
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Service;

@Service
public class LocationConsumer {

    @KafkaListener(topics = "delivery-boy-locations", groupId =
"end-user-group")
    public void listen(String message) {
        System.out.println("Received: " + message);
        // In a real app, update the UI/map here
    }
```

1. }

Testing the App with 200K Messages

1. **Start Kafka and Zookeeper** (as above).
2. **Create the Topic**:
   text
   CollapseWrapCopy
   ```
   bin/kafka-topics.sh --create --topic delivery-boy-locations
   --bootstrap-server localhost:9092 --partitions 3 --replication-factor 1
   ```
3. **Run Both Apps**:
   - Start EndUserApp (consumer) first.
   - Start DeliveryBoyApp (producer).
4. **Simulate 200K Messages**:

Modify LocationController to send messages in a loop:
java
CollapseWrapCopy

```java
@GetMapping("/simulate")
public String simulate() {
    for (int i = 0; i < 200000; i++) {
        double lat = 12.9716 + (i * 0.0001); // Simulate movement
        double lon = 77.5946 + (i * 0.0001);
        producer.sendLocation("D1", lat, lon);
    }
    return "Simulation complete!";
```

- }

- Hit http://localhost:8080/simulate in a browser or Postman.

5. **Observe**:
    - Check the EndUserApp console—it will print all 200K messages as they arrive.
    - For a real app, you'd process these (e.g., update a map) instead of just printing.

Summary for Beginners

- **Kafka**: A tool for handling real-time data streams, like a delivery boy's location updates.
- **Why Not Database**: Databases can't handle high-speed, continuous data as efficiently.
- **Zomato Use Case**: Kafka connects the delivery boy's app to the end user's app seamlessly.
- **Setup**: Start Zookeeper and Kafka, create topics, and use Spring Boot to produce/consume messages.
- **Test**: Simulate 200K messages to see Kafka's power.

This is a basic setup—Kafka can scale much further with multiple brokers and partitions in production! Let me know if you need help with any step!

so what is the role of db have what info the database wil store? or when the database is updated?

Role of the Database

The database complements Kafka by storing **static or semi-static data** that doesn't need real-time streaming but is critical for the application's functionality. It acts as the **source of truth** for information that isn't transient (like live location updates).

What Information Does the Database Store?

In the Zomato live location tracking use case, the database might store:

1. **User Data**:
    - Customer details (name, phone, address).
    - Delivery boy details (ID, name, contact info).
2. **Order Data**:
    - Order ID, customer ID, delivery boy ID, restaurant details, items ordered, order status (e.g., "Placed," "In Transit," "Delivered"), and timestamps (e.g., order placed time, estimated delivery time).
3. **Historical Data**:

    - Past orders and their final delivery locations (not live updates, but the end result).
    - Delivery boy performance metrics (e.g., average delivery time).
4. **Configuration Data**:
    - Restaurant locations, delivery zones, or pricing rules.

Example Database Schema (Simplified):

Orders Table:
text
CollapseWrapCopy
```
order_id | customer_id | delivery_boy_id | restaurant_id | status       |
placed_time

---------+-------------+-----------------+---------------+--------------+-----
--------
```

- `O123 | C001 | D1 | R001 | In Transit
  | 2025-03-18 10:00:00`

Delivery Boys Table:
text
CollapseWrapCopy
```
delivery_boy_id | name       | phone

----------------+------------+-------
```

- `D1              | John Doe   | 1234567890`

Customers Table:
text
CollapseWrapCopy
```
customer_id | name      | address

------------+-----------+---------
```

- `C001        | Jane Smith| 123 Main St`

When Is the Database Updated?

The database is updated when there's a **significant state change** or when data needs to be persisted for long-term use, not for every fleeting live update (that's Kafka's job). Here's when and how it might happen in the Zomato example:

1. **Order Creation**:
    - When a customer places an order, the database creates a new row in the Orders table with status "Placed."
2. **Order Assignment**:
    - When a delivery boy is assigned, the delivery_boy_id is updated in the Orders table.
3. **Status Changes**:
    - When the delivery boy picks up the food (status → "In Transit") or delivers it (status → "Delivered"), the database updates the status and possibly adds a delivered_time.
4. **Post-Delivery**:
    - After delivery, the final location or order summary might be stored for historical records.
5. **User Updates**:
    - If a customer updates their address or a delivery boy's info changes, the respective tables are updated.

Key Point:

- The database isn't updated for every live location ping (e.g., every 5 seconds from the delivery boy). That's too frequent and unnecessary for long-term storage—Kafka handles that transient data.

How Kafka and the Database Work Together

Kafka and the database complement each other in a real-time system:

1. **Kafka's Role**:
    - Handles **real-time, transient data** like the delivery boy's live location updates ({deliveryId: "D1", lat: 12.9716, lon: 77.5946}).
    - Messages are produced and consumed instantly, and Kafka stores them temporarily (e.g., for 7 days) based on retention settings.
2. **Database's Role**:
    - Stores **persistent, structured data** like order details, user info, and final outcomes.
    - Updated only when there's a meaningful event (e.g., order status change), not for every location ping.

Zomato Flow Example:

- **Order Placed**: Customer places an order → Database creates an order record (status: "Placed").
- **Delivery Boy Assigned**: Database updates delivery_boy_id and status to "Assigned."
- **Live Tracking Starts**: Delivery boy's app sends location updates to Kafka → End user's app consumes them to show the live map.
- **Order Delivered**: Delivery boy marks order complete → Database updates status to "Delivered" and logs the final timestamp.

Why Not Store Live Location in the Database?

Storing every location update (e.g., 2,000 updates/second for 10,000 deliveries) in a database would:

- Overwhelm the database with constant writes.
- Slow down queries due to the massive data volume.
- Be unnecessary since live locations are only relevant in real time and don't need long-term storage.

Instead:

- Kafka streams these updates efficiently.
- The database only stores the final delivery location (if needed) once the order is complete.

Theoretical Integration in the Spring Boot Project

In the earlier Spring Boot example:

- **DeliveryBoyApp**: Produces live location messages to Kafka.
- **EndUserApp**: Consumes them to show the map.
- **Database**: Could be added to store order details and update them via a separate service.

Modified Flow:

1. **Order Service** (new microservice):
    - Creates an order in the database (e.g., MySQL).
    - Assigns a delivery boy and updates the database.
2. **DeliveryBoyApp**:
    - Queries the database for active orders assigned to the delivery boy.
    - Sends location updates to Kafka for those orders.
3. **EndUserApp**:

    - Queries the database for the customer's active order details (e.g., delivery boy ID).
    - Subscribes to Kafka for location updates matching that delivery boy ID.
4. **Order Completion**:
    - DeliveryBoyApp sends a "Delivered" event to Kafka.
    - A listener updates the database with the final status.

Summary

- **Database Stores**: Persistent data (orders, users, historical records).
- **Database Updates**: On significant events (order creation, status changes), not real-time pings.
- **Kafka Handles**: Real-time, high-frequency data (live locations).
- Together, they create a system where Kafka manages the fast-moving data, and the database keeps the stable, long-term records.

Does this clarify the database's role? Let me know if you want to dive deeper into integrating it with the Spring Boot apps!

how object looks like then how serialized string loooks like explain with one example all

Example: A "Person" Object

Imagine we're working with a "person" in a program (I'll use JavaScript as an example since it's common and easy to understand).

1. How the Object Looks

An object is like a container that holds information. In a program, it's written with properties (like name and age) and their values. Here's what it looks like:

javascript

CollapseWrapCopy

```
let person = {
  name: "Sarah",
  age: 28,
  city: "New York"
```

```
};
```

- **What this means**: This is an object called person. It has three pieces of info:
  - name is "Sarah" (a string/word),
  - age is 28 (a number),
  - city is "New York" (another string).
- **How it's stored**: The computer keeps this in memory as a structured "thing" you can use. You could ask it, "What's the person's name?" and it'd say "Sarah."

2. How the Serialized String Looks

Serialization turns this object into a flat string so it can be saved (like in a file) or sent (like over the internet). In this case, we'll serialize it into JSON (a popular format). Here's what happens:

javascript

CollapseWrapCopy

```
let serializedPerson = JSON.stringify(person);
```

- **Resulting string**:

text

CollapseWrapCopy

```
{"name":"Sarah","age":28,"city":"New York"}
```

- **What this means**:
  - The object is now a single line of text.
  - The curly braces {} show it's an object.
  - Each property (name, age, city) is followed by a colon : and its value.
  - Strings (like "Sarah" and "New York") get quotation marks " ".
  - Numbers (like 28) don't need quotes.
  - Everything is squished together with commas , between properties.
- **Why it's useful**: This string can be saved to a file, sent in an email, or posted online, and it's still readable by humans and computers.

Putting It All Together

Object (in memory):
text
CollapseWrapCopy
```
{
```

```
name: "Sarah",

age: 28,

city: "New York"
```

- }
    - Looks neat and structured, like a little box with labeled slots.
    - You can easily grab person.name and get "Sarah".
- **Serialized String (as text)**:
  text
  CollapseWrapCopy
  ```
  {"name":"Sarah","age":28,"city":"New York"}
  ```
    - Looks like one long line of text.
    - No spaces between parts (to save space), but still clear enough to understand.

Bonus: Deserialization

If you get that string back and want to use it as an object again, you deserialize it:

javascript

CollapseWrapCopy

```
let personAgain = JSON.parse('{"name":"Sarah","age":28,"city":"New York"}');
```

- Now personAgain is an object just like the original, and you can ask for personAgain.name to get "Sarah".

So, the object is like a living thing in your program, while the serialized string is like a postcard version of it—flat, portable, and ready to travel! Does this example help?

Object Creation

- **What it is**: Making a new "thing" in your program that can hold information.
- **Easy example**: Think of it like making a new contact in your phone. You create a "person" and give it a name and number.
- **How it works**: You tell the computer, "Hey, make me a new object called 'person' and let it have a name like 'John' and age of 30."

Conversion

- **What it is**: Changing one type of information into another type.
- **Easy example**: Turning the number 5 into the word "5" so you can add it to other words.
- **How it works**: Sometimes the computer does it automatically (like adding a number to a word), or you tell it exactly how to change it (like saying "turn this number into a word").

Serialization

- **What it is**: Turning an object into a simple string so you can save it or send it somewhere.
- **Easy example**: Imagine taking your phone contact and writing it on paper as "name:John,age:30" to mail to a friend.
- **How it works**: The computer takes your object (like a person with a name and age) and turns it into a string, like {"name":"John","age":30}, so it's easy to store or share.

Deserialization

- **What it is**: Taking that string and turning it back into an object you can use.
- **Easy example**: Your friend gets the paper with "name:John,age:30" and puts it back into their phone as a contact.
- **How it works**: The computer reads the string {"name":"John","age":30} and rebuilds it into an object with a name and age you can work with again.

In short:

- **Creation**: Make a new thing.
- **Conversion**: Change its type.
- **Serialization**: Turn it into a string to save or send.
- **Deserialization**: Turn the string back into a thing.